FRONTEND DEVELOPMENT BOOTCAMP 2026

Session 7

JavaScript Interactivity & DOM Basics

Functions · Arrays · DOM Selection & Manipulation · Events · Interactive Elements

3:00 PM – 5:00 PM

Live on Zoom

Minimum 2 hours

Speaker

Utkarsh Kushwaha

ABOUT THE SPEAKER

Utkarsh Kushwaha

Full Stack Developer
Technical Speaker
Community Mentor

Experienced in building responsive and interactive web applications using modern technologies and actively involved in technical communities, workshops, and student learning initiatives.

Full-Stack Developer

Building AI-powered full-stack projects.

TODAY'S SYLLABUS

01
Functions
Declarations, parameters, return values, arrow functions

02
Arrays
Creating, accessing, looping & common array methods

03
Introduction to the DOM
What the DOM is & how the browser builds it

04
DOM Selection & Manipulation
querySelector, changing text, styles & attributes

05
Events & Event Handling
addEventListener, click/input/submit events

06
Interactive Web Elements
Building a mini interactive UI with everything combined

TOPIC 01 · FUNCTIONS

Functions — Reusable Blocks of Logic

- A function packages code you want to run more than once.
- Declare with function name(params) { ... }.
- Arrow functions: const name = () => { ... } — shorter syntax.
- return sends a value back to wherever the function was called.

functions.js

```
// function declaration
function greetStudent(name) {
  return `Hello, ${name}! 👋`;
}

// arrow function
const add = (a, b) => a + b;

console.log(greetStudent("Utkarsh"));
```

Real-World Example

An e-commerce site uses a calculateTotal(cartItems) function every single time you add or remove a product — instead of rewriting the price-adding logic on every button.

cart.js

```
function calculateTotal(cart) {
  let total = 0;
  for (let item of cart) {
    total += item.price * item.qty;
  }
  return total;
}

calculateTotal(cart); // ₹2,450
```

TOPIC 02 · ARRAYS

Arrays — Storing Lists of Data

- An array holds an ordered list of values in one variable.
- Access items by index: arr[0] is the first item.
- Loop with for, for...of, or .forEach() to visit each item.
- .push(), .pop(), .map(), .filter() are common built-in methods.

arrays.js

```
const students = ["Aman", "Riya", "Sam"];

students.push("Neha");
console.log(students[0]);  // "Aman"

students.forEach((s) => {
  console.log(`Hi ${s}`);
});
```

Real-World Example

A to-do list app stores every task inside a tasks array. Adding a task pushes to it, completing one filters it — the array is the single source of truth for what renders on screen.

todo.js

```
let tasks = ["Learn DOM", "Build UI"];

// add a new task
tasks.push("Submit assignment");

// remove a completed task
tasks = tasks.filter(
  (t) => t !== "Learn DOM"
);
```

TOPIC 03 · THE DOM

What is the DOM?

Document Object Model — the browser's live, tree-shaped representation of your HTML that JavaScript can read and change.

It's a tree

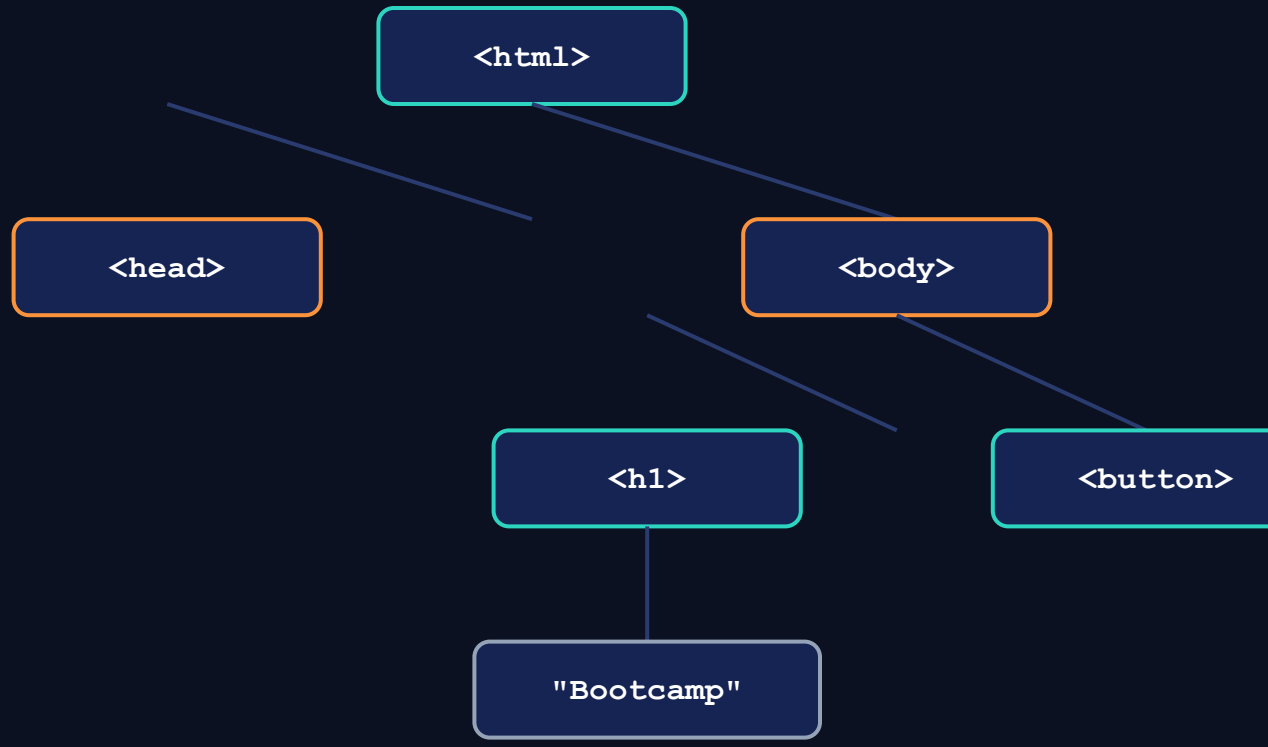
Every HTML tag becomes a connected "node".

Browser builds it

Created automatically when a page loads.

JS can change it

Update it live — no page reload needed.



Every box above is a "node" — JavaScript can select any node and read or change what's inside it, live in the browser.

TOPIC 04 · DOM SELECTION & MANIPULATION

Selecting & Changing Elements

- document.querySelector(".class") — grabs the first match.
- document.querySelectorAll("li") — grabs all matches as a list.
- .textContent / .innerHTML — read or change what's inside.
- .style.property — change CSS directly from JS.
- .classList.add/remove/toggle() — control classes dynamically.

select.js

```
const title = document
  .querySelector("h1");

title.textContent = "Welcome!";
title.style.color = "#2DD4BF";
```

Real-World Example

A dark-mode toggle on a website selects the <body> element and swaps its class — every color on the page updates instantly, no reload.

darkmode.js

```
const body = document.body;
const btn = document
  .querySelector("#themeBtn");

btn.addEventListener("click", () => {
  body.classList.toggle("dark");
});

// .dark { background: #0B1120; }
```

TOPIC 05 · EVENTS & EVENT HANDLING

Listening for User Actions

- An event is anything the user does: click, type, scroll, submit.
- element.addEventListener("click", handlerFn) listens for it.
- The handler function runs automatically when the event fires.
- event.preventDefault() stops a form's default page reload.

events.js

```
const btn = document
  .querySelector("#likeBtn");

btn.addEventListener("click", () => {
  console.log("Button clicked!");
});
```

Real-World Example

Instagram's like button listens for a click event, instantly turns red, and updates the like count — all through one event listener.

form.js

```
const form = document
  .querySelector("#loginForm");

form.addEventListener("submit", (e) => {
  e.preventDefault();
  console.log("Form submitted!");
});
```

Putting It All Together

Functions + Arrays + DOM + Events = a real interactive feature. Let's build a simple task counter.

taskCounter.js

```
// Select Elements
const heading =
document.querySelector("#heading");
const box = document.querySelector("#box");
const button = document.querySelector("#btn");

// Button Click Event
button.addEventListener("click", () => {

    // Change the text
    heading.textContent = "Welcome to
JavaScript DOM!";

    // Change the color of the box
    box.style.backgroundColor = "tomato";

});
```

What's happening here?

Array

tasks[] stores every task the user adds.

DOM Selection

querySelector grabs the input, button & list.

Function

renderTasks() redraws the list on every change.

Event

A click event triggers the whole update chain.

This is the pattern behind to-do apps, cart counters & like buttons.

RECAP

Key Takeaways from Today

Functions

Package reusable logic with function or arrow syntax.

Arrays

Store & loop through lists of data with built-in methods.

The DOM

The tree-structure the browser builds from your HTML.

Selection

querySelector() / querySelectorAll() find any element.

Manipulation

Change text, styles & classes to update the page live.

Events

addEventListener() reacts to clicks, input & submits.

Thank You